

1.null

`null`是主动赋值空值，`undefined`是为定义

TDZ

`let, const, class, import`生命前不能访问
`function, function*, var`

2.eventloop

宏任务

`setTimeout, setInterval, setImmediate, requestAnimationFrame,`

微任务

`await`后，`.then`，`process.nextTick`

宏任务微任务

区分任务的紧急程度

避免单任务卡死主线程，插入渲染机会

渲染机会

每个宏任务执行完成后，计算布局绘制合成

`raf`

主要功能是让样式的渲染和刷新率同步

聚焦渲染的时机选择

`script`宏任务 → 同步任务 → 微任务 → 【样式计算 → 布局 → 绘制 → 合成】 → 渲染到屏幕
↑
`raf` 在这里执行

3Promise

Promise

`pending, fulfilled, rejected`

Promise就是一个任务

创建之后就开始执行，然后对应有三个状态，

成功后在`.then`中获取对应成功结果

失败在`catch`获取失败结果

4generator

Iterator

通过next()方法

Generator

function*配合yield的循环方法

```
function* lazyRange(max) {  
  for (let i = 0; i < max; i++) {  
    yield i;  
  }  
}  
  
const lazyNums = lazyRange(1000000); // 几乎不占内存  
console.log(lazyNums.next().value); // 需要时才计算
```

5.sse

用TextDecoder解码收到的数据

开启stream流，避免单个字符被切割乱码

用buffer来限制在\n分割，来避免一句话是细碎

6.sse-md

md中，完整单元是跨行的，所以buffer截断\n还是会有

代码没闭合、chunk解析全文的问题

用状态机解决块级闭合

识别```，设置状态处理

实际现在使用主流插件

解析全文

不处理全部内容，只处理增量的内容

7sse-websocket

sse单向，对话

websocket双向，实时对话

空闲断开，增加心跳检测（30s ping一下）

8继承和原型链

访问对象的属性时

JS 会先在对象自身查找，找不到就去原型上找，还找不到就去原型的原型上找.....

直到找到或到达 null

```
// ❌ 经典问题
function Gadget() {}
Gadget.prototype.config = { theme: 'dark' };

const g1 = new Gadget();
const g2 = new Gadget();

g1.config.theme = 'light';
console.log(g2.config.theme); // 'light' (被意外修改了!)
```

9事件冒泡捕获

冒泡由内向外

捕获由外向内

```
<div id="parent">
  <button id="child">点我</button>
</div>

<script>
  const parent = document.getElementById('parent');
  const child = document.getElementById('child');

  // 捕获阶段监听（从外向内）
  parent.addEventListener('click', () => console.log('parent 捕获'), true);
  child.addEventListener('click', () => console.log('child 捕获'), true);

  // 冒泡阶段监听（从内向外）
  parent.addEventListener('click', () => console.log('parent 冒泡'), false);
  child.addEventListener('click', () => console.log('child 冒泡'), false);

  // 点击按钮后的输出顺序：
  // parent 捕获    (1)
  // child 捕获     (2)
  // child 冒泡     (3)
  // parent 冒泡    (4)
</script>
```

10闭包

闭包是不是比如
我用`setTimeout`输入一个`value`
然后我修改了`value`，
实际`setTimeout`中的`value`已经是闭包了，所以不是输出修改后的值

11.浏览器全过程

输入 URL 到展示页面过程：DNS → TCP握手 → HTTP请求 → 响应 → DOM树 → CSSOM → 渲染树 → 布局 → 绘制 → 合成

css放headqian, script放body后且async

async 下载完立即执行顺序不确定, defer 等 DOM 解析完按序执行

12status code

2xx 200-299 成功 搞定了
200, 201, 204, 206
3xx 300-399 重定向 去别处找
301, 302, 304 (缓存)
4xx 400-499 客户端错误 你搞错了
401未授权, 403禁止, 404, 405
5xx 500-599 服务端错误 我挂了
500, 501, 502Gateway,

13浏览器安全

跨域
浏览器限制不同网址不能发送请求
避免恶意网址发送请求时携带浏览器的cookie
nginx不受浏览器限制, 无跨域问题
jwt token
header、payload、signature

14refreash token

核心
isRefreashing锁, 防止并发刷新
failedQueue, 存重试的请求

15溢出

单行溢出
white-space:nowrap
overflow:hidden
text-overflow:ellipsis;
多行
webkit

16vue

核心

vue2基于defineProperty遍历所有属性的getter来更新

vue3基于proxy代理整个对象，在访问时才代理深层对象

vue3-proxy原理

1.handler拦截进行 读写删

2.const state = reactive (...), 此时只有state有proxy, 内部字段无proxy

内部字段被使用时, 才通过handler.get获取字段对象, 并解析、创建proxy, 返回对应proxy

3.vue3的数组带索引的追踪, 可以用effect/watch追踪

```
const arr = reactive([1, 2, 3])
```

```
effect(() => {
```

```
  console.log(arr[0])  // 自动收集依赖
```

```
})
```

```
arr[0] = 100  // 自动触发更新
```

17watch/effect/computed

effect

监听访问到的属性

```
const stop = effect(() => {  
  console.log('effect 执行:', state.count)  
})
```

watch

监听指定的属性

加deep, 可以监听到对象内部属性

```
watch(user, (newVal, oldVal) => {  
  console.log(newVal?.name)  
})
```

watchEffect

```
watchEffect(() => {  
  console.log(`count: ${count.value}`)  
  console.log(`name: ${name.value}`)  
  console.log(`age: ${state.age}`)  
  // 没有访问 city, 所以 city 变化不会触发  
})
```

watch监听指定属性

effect监听访问属性

watchEffect基于effect封装, 在组件写在时自动停止

watchEffect基于effect封装, 实现组件卸载时自动停止监听, 并且有额外的onCleanup、onTrack

18filters/computed/method

```
computed
  const count = ref(0)
  const double = computed(() => {
    console.log('计算 double')
    return count.value * 2
  })
```

19vue-diff

virtual DOM，然后和旧树对比，只更新有变化的部分，并且这只进行同层级的对比

然后对比方法有基本的双端对比，就是新旧的头尾进行比较

然后vue3给dom加了key

静态提升

`<h1>静态文本</h1>` 这是静态的，用js获取dom进行修改会被覆盖
`<h1>{{响应式文本}}</h1>` 改为响应式，升级为动态的

20vite/webpack-esm/commonjs

diff

指的是编译时的更新

vite单文件替换，局部更新，更快：

webpack链路遍历定位更新，更慢

编译时：

vite

基于esm (import/export)，

import了文件的什么方法就编译什么，按需编译

启动时不编译，加载是在浏览器运行时，文件import了什么编译什么

webpack

启动时全量编译，生成bundle给浏览器运行

treeshaking

esm的import保证了按需引入，

进而保证了treeshaking可以根据import的内容在打包时去除无用的东西

总体来说：

vite-rollup打包比webpack快，和启动时的原理是一样的，

都是基于import按需编译/打包，

webpack都会先全量打包/编译

21缓存更新

核心:

有contentHash和CacheControl两个功能链路

目的都是同步浏览器和服务器的缓存

contentHash

文件名带上hash，前端文件更新了，文件名基于hash也更新

浏览器判断这个文件名是全新的，立即向服务器请求同步缓存

CacheControl

根据缓存策略（max-age/no-cache）判断是否缓存过期

缓存过期，向服务器请求同步缓存

22css

水平居中

text-align:center 内联元素，行内块

margin:auto 块级元素，有宽度

flex justify-content

grid justify-items

position: absolute; left: 50%; transform: translateX(-50%)

垂直居中

flex text-align

grid text-align

line-height 单行文本

top: 50%; transform: translateY(-50%)

box-sizing: border-box;

width = 内容 + padding + border

更符合直觉，布局更可控

flex布局

flex-direction\justify-content\align-items\flex-wrap\flex

单位

px 像素

rem 根元素font-size

em 父元素font-size

vm/vh 视窗的高宽

% 父元素对应属性

position

实际常用的就是relative搭配absolute使用，

或者fixed实现悬浮

或者sticky实现通讯录字母索引、顶部导航栏（不会脱离文档流，让文档流这部分空给sticky）

23CLS/LCP

Latgest Contentful Paint最大内容绘制

最大内容加载时间、图片、视频、文本

优化服务器响应、渲染阻塞的js, css

优化图片资源

Cumulative Layout Shift累计布局偏移

页面互相挤压乱动

优化图片尺寸、使用骨架屏、动画避免位移

24伪类/伪元素

: 伪类

:: 伪元素

```
a:link { color: blue; } /* 未访问 */
```

```
a:visited { color: purple; } /* 已访问 */
```

```
a:hover { color: red; } /* 鼠标悬停 */
```

```
a:active { color: green; } /* 点击瞬间 */
```

25

virtual table

计算可视区域，监听滚动事件，动态渲染